Python 函数基础

入门讲义与配套练习

学习目标：只掌握函数的最基础概念与最基本应用。

覆盖：函数定义与调用、形参与实参、返回值、docstring、嵌套调用、简单案例、局部/全局变量、`global`。

题量：10 单选 + 5 多选 + 5 填空 + 4 基础解答 = 24 题

难度：入门 / 基础

核验日期：2026-08-20

使用说明

本资料严格围绕上传图片中的 Python 函数知识组织。正文先进行图片内容重新编码，再对明显笔误、语法问题和概念过度简化之处逐项纠正，最后给出极低难度练习与集中解析。网络核验优先使用 Python 官方文档、PEP、MIT OpenCourseWare、UC Berkeley CS 61A 与 Sphinx 官方文档。[1][2][4][5][7][8]

本资料的边界

本资料不展开默认参数、关键字参数、可变参数、`lambda`、闭包、`nonlocal`、装饰器、递归、生成器等进阶内容。出现这些概念时，只作必要边界说明，不把它们设计为练习核心考点。

目录

使用说明 **1**

1 图片内容重新编码 **3**

1.1 第 1 页：3.4 函数基础——函数定义、调用、参数与返回值 3

1.2 第 2 页：多个参数、函数说明文档、嵌套调用 4

1.3 第 3 页：嵌套调用执行顺序与两个基础案例 5

1.4 第 4 页：成绩案例续写、变量作用域与 global 6

2 原图问题与必要纠错 **6**

3 知识点清单 **8**

4 核心概念详解 **8**

4.1 函数是什么 8

4.2 参数：形参与实参 9

4.3 return 返回值 9

4.4 函数说明文档 docstring 10

4.5 圆面积函数中的公式 10

4.6 函数嵌套调用 11

4.7 案例 1：统计字符串中的元音字母 11

4.8 案例 2：成绩最高分、最低分、平均分 12

4.9 变量作用域：局部变量与全局变量 13

4.10 global 关键字 13

5 高频易错点总结 **14**

6 知识点—题目对应表 **14**

7 配套练习 **16**

7.1 单项选择题（10 题） 16

7.2 多项选择题（5 题） 18

7.3 填空题（5 题） 19

7.4 基础解答题（4 题） 19

8 答案速查表 **20**

9 单项选择题详细解析 **20**

10 多项选择题详细解析 **22**

11 填空题详细解析 **24**

12 基础解答题详细解析 **25**

13 基础解题步骤总结 **27**

14 资料核验说明 **28**

15 参考资料与核验来源 **28**

16 质量检查记录 **29**

1 图片内容重新编码

1.1 第 1 页：3.4 函数基础——函数定义、调用、参数与返回值

原图正文的规范化转录

Python——第一章核心语法

3.4 函数基础

一、函数定义

1. 定义：函数是组织好的、可重复使用的、用于实现特定功能的代码片段。

2. 函数的定义与调用：

```
def function_name(parameter_list):
    function_body
    ...
    return return_value

# call a function
function_name(arguments)
```

3. 注意：函数定义时的参数列表与返回值语句可以有，也可以没有；原图写有“函数必须先定义后调用执行”。

4. 示例代码：

```
def out_line():
    print("----------")

out_line()
```

二、函数的参数与返回值

```
def function_name(parameter_list):
    function_body
    ...
    return return_value

function_name(arguments)
```

1.2 第 2 页：多个参数、函数说明文档、嵌套调用

原图正文的规范化转录

补充注意：函数定义时如果有多个参数，多个参数之间使用逗号分隔。

三、函数说明文档

1. 定义：写在函数开头、用三个引号包裹的字符串，用于解释函数的功能、参数、返回值等信息，方便调用者了解函数的具体作用。
2. 原图示例：

```
def circle_square(r):
    """
    该函数用于计算给定半径圆的面积
    :param r: 圆的半径
    :return: 圆的面积
    """
    return 3.14 * r * r
```

四、函数的嵌套调用

1. 含义：在一个函数中又调用另外一个函数。
2. 特点：原图写为“函数调用遵循栈结构，最后被调用的函数最先返回”。
3. 示例代码：

```
def function_a():
    print("a... before")
    function_b()
    print("a... after")

def function_b():
    print("b... before")
    function_c()
    print("b... after")

def function_c():
    print("c...")
```

1.3 第 3 页：嵌套调用执行顺序与两个基础案例

原图正文的规范化转录

原图继续给出：

```
function_a()
```

并用树状流程表示：先打印 a... before，进入 function_b()；再打印 b... before，进入 function_c()；打印 c... 后返回 function_b()，再打印 b... after；最后返回 function_a()，打印 a... after。

五、案例

案例 1：定义一个函数，计算传入字符串中元音字母的个数。

```
def count_aeiou(string):
    num = 0
    for word in string:
        if word in "aeiouAEIOU":
            num += 1
    return num
```

案例 2：原图题意中有一小段手写文字无法完全可靠辨认；可以可靠确认的核心要求是：**传入一个成绩列表，计算最高分、最低分和平均分，并让平均分保留 1 位小数**。

```
def calc_score(score_list):
    """
    计算传入成绩列表中成绩的最高分、最低分以及平均分
    """
```

1.4 第 4 页：成绩案例续写、变量作用域与 global

原图正文的规范化转录

成绩案例后续：

```
# :param score_list: 分数列表
# :return: 最高分、最低分、平均分
max_s = max(score_list)
min_s = min(score_list)
avg_s = round(sum(score_list) / len(score_list), 1)
return max_s, min_s, avg
```

原图红笔注明“保留 1 位小数”。

六、函数进阶

1. 变量作用域

1. 含义：变量的作用域指变量的作用范围。
2. 分类：全局变量、局部变量。
   (a) 全局变量：原图写为“在函数之外定义的变量，称之为全局变量，在整个文件都可以用”。
   (b) 局部变量：原图写为“在一个函数内部定义的变量，只能在该函数内部使用，外部不可访问”。
3. `global` 关键字：原图说明其用途为在函数中使用全局变量，使得可以在函数内部修改全局变量。

原图语法示例手写为：

```
def circle_square(r):
    global pi = 3.14
    square = pi * r * r
    return square
```

并注明：`global` 主要用于程序状态、配置、计数器等场景中。

2 原图问题与必要纠错

序号	原图写法或问题	规范修正与理由
1	“多个参数之间使用逗号分割”	应写为“多个参数之间使用逗号**分隔**”。这是中文用词笔误，不影响 Python 语法。
2	“先定义后调用”	作为入门记忆基本正确，但更精确地说：程序执行到函数调用时，该函数名必须已经绑定到函数对象。`def` 执行后才创建并绑定函数对象。[1]

序号	原图写法或问题	规范修正与理由
3	参数与返回值“可有可无”	正确。函数可零参数；函数没有显式 `return` 时仍会返回 `None`。[1]
4	“函数说明文档就是三引号字符串”	需要补充：要成为函数的 `__doc__`，文档字符串必须是函数体中的**第一条语句**。PEP 257 推荐使用三重双引号。[1][4]
5	`:param`、`:return`	这些不是 Python 语法关键字，而是 reStructuredText/Sphinx 常见信息字段写法。普通 docstring 不要求必须使用这种格式。[8]
6	`return 3.14 * r * r`	语法正确，适合基础练习。数学上标准公式是 $S = \pi r^2$。若追求更高精度，通常可使用 `math.pi`；本资料为避免超出图片范围，练习仍允许使用 3.14。
7	“函数调用遵循栈结构，最后被调用的函数最先返回”	可作为普通同步嵌套函数的入门模型理解。MIT 与 Berkeley 的课程材料都使用 frame / environment / stack 等模型解释函数调用和作用域。[6][7]
8	成绩函数中定义 `avg_s`，却写 `return ..., avg`	这是可运行性错误：`avg` 未定义，会产生 `NameError`。应改为 `return max_s, min_s, avg_s`。
9	成绩列表没有说明是否允许为空	`max([])`、`min([])` 会报错，且平均值会出现除以零。因此基础版本应明确前提：`score_list` 为非空数值列表。[2]
10	“全局变量在整个文件都可以用”	过度简化。更准确地说：模块顶层定义的名字属于模块全局作用域；函数内部读取全局名通常不需 `global`，但若要在函数内部给该名字重新赋值并让赋值作用于全局绑定，则需要 `global`。[3]
11	`global pi = 3.14`	这是**非法语法**。`global` 语句只声明名字，例如 `global pi`，不能与赋值写在同一条语句。[3]
12	圆面积函数只“读取”全局 `pi` 却使用 `global`	如果函数仅读取全局 `pi`，其实不需要 `global`；只有在函数内部希望对全局名字重新绑定时才需要。[3]

原图中无法可靠辨认的内容

第 3 页“案例 2”的题干中，成绩列表前有一小段手写修饰语无法从照片中完全可靠辨认。本资料不把这段文字当作考点，只保留可以确定的任务核心：**输入成绩列表，求最高分、最低分和平均分（保留 1 位小数）**。

3 知识点清单

编号	知识点	最低掌握要求
K1	函数的作用	知道函数用于封装可重复执行的功能。
K2	`def` 定义函数	能写出 `def name(...):` 并正确缩进函数体。
K3	调用函数	能写出 `name(...)`。
K4	形参与实参	知道定义处是形参，调用处传入的是实参。
K5	`return`	知道它结束当前函数并把结果交给调用者；没有显式返回值时得到 `None`。
K6	docstring	知道它是函数体第一条字符串语句，用来说明功能、参数、返回值。
K7	嵌套调用	能按“进入被调函数—返回原函数”的顺序判断简单输出。
K8	简单统计案例	会用循环统计元音；会用 `max()`、`min()`、`sum()`、`len()`、`round()` 求成绩统计量。
K9	局部与全局作用域	知道函数内赋值通常形成局部变量，模块顶层变量属于全局作用域。
K10	`global`	会写 `global name`，并知道仅在需要修改全局绑定时使用。

4 核心概念详解

4.1 函数是什么

定义

函数（function）是一段具有名字的可调用代码，用于把一组操作封装起来，便于重复使用和组织程序。Python 官方教程指出，`def` 关键字引入函数定义，后面跟函数名与括号中的形参列表，函数体从下一行开始并必须缩进。[1]

通俗理解：把一段经常要做的操作装进一个“工具盒”，给工具盒取名。以后需要时只要“叫它的名字”，而不用重复写所有步骤。

最基础使用步骤：

1. 用 `def` 写函数头；
2. 写冒号 `:`；
3. 下一行缩进写函数体；
4. 需要结果时用 `return`；
5. 在函数定义执行之后，用 `函数名（...）` 调用。

极简单示例

```
def say_hi():
    print("Hi")

say_hi()
```

输出：Hi。

常见错误：漏冒号、函数体不缩进、把函数定义和函数调用混为一谈。

一句记忆：def 是“造工具”，括号调用是“用工具”。

4.2 参数：形参与实参

定义

形参（formal parameter）写在函数定义的括号中，相当于函数内部接收输入的“名字”；实参（argument）是在调用函数时真正传进去的值。官方教程把 def 后括号中的列表称为 formal parameters。[1]

```
def add(a, b):      # a and b are parameters
    return a + b

result = add(2, 3)  # 2 and 3 are arguments
```

多个参数的最基本规则：用英文逗号 , 分隔，例如 def add(a, b):。

常见错误：

- 忘记逗号；
- 定义需要两个参数，却调用时只传一个值；
- 把“参数名”与“传入的具体值”混为一谈。

一句记忆：定义处是“占位名字”，调用处是“真正数据”。

4.3 return 返回值

核心规则

return expression 会结束当前函数调用，并把表达式的值作为返回值交给调用者。return 后没有表达式，或者函数执行到末尾都没有遇到 return，返回值都是 None。[1]

```
def square(x):
    return x * x

y = square(3)
print(y)
```

这里 x=3，函数计算 3*3=9，return 9 把 9 交给调用处，所以 y 得到 9。

print 与 return 不一样

`print()` 是把内容显示到屏幕；`return` 是把值交回调用者。MIT 的入门课程专门区分了二者：返回值可以继续参加表达式计算，而 `print()` 的主要作用是产生输出副作用。[5][6]

一句记忆：print 给人看，return 给程序用。

4.4 函数说明文档 docstring

定义

函数体的第一条语句如果是字符串字面量，它会成为该函数的文档字符串（docstring）。PEP 257 推荐用三重双引号书写，并建议函数文档说明行为、参数、返回值、副作用、异常和限制等必要信息。[1][4]

```
def circle_area(r):
    """Return the area of a circle with radius r."""
    return 3.14 * r * r
```

图片中使用的 `:param r:` 与 `:return:` 是 Sphinx/reStructuredText 常见字段，可被文档工具结构化展示；它们不是 Python 语法的强制要求。[8]

一句记忆：docstring 是“写给调用者看的函数说明书”。

4.5 圆面积函数中的公式

圆面积公式为

$$S = \pi r^2.$$

其中：

- S：圆的面积，是要求的结果；
- π：圆周率，图片中用 3.14 近似；
- r：圆的半径，是函数输入；
- $r^2 = r \times r$。

适用条件：输入的 r 应表示半径，通常要求 $r \geq 0$。如果半径有长度单位，例如 cm，那么面积单位就是 cm^2。

半径 2 的最基础代入

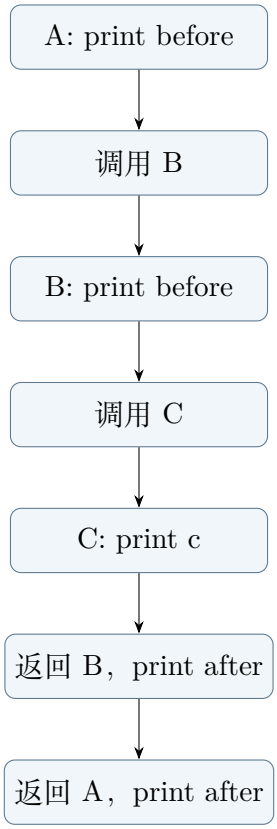
若 $r = 2$，使用图片中的 $\pi \approx 3.14$：

$$S = 3.14 \times 2 \times 2 = 12.56.$$

因此 `circle_area(2)` 返回 `12.56`。

4.6 函数嵌套调用

当函数 A 的函数体中调用函数 B，就产生了最基础的嵌套调用。可以用“暂时离开 A，进入 B；B 完成后回到 A 继续”的方式理解。



因此原图代码运行顺序是：

```
a... before
b... before
c...
b... after
a... after
```

一句记忆：调用时先“进去”，返回后再“接着往下”。

4.7 案例 1：统计字符串中的元音字母

```
def count_vowels(string):
    num = 0
    for ch in string:
        if ch in "aeiouAEIOU":
            num += 1
    return num
```

逐行理解：

1. `num = 0`：准备一个计数器；

2. `for ch in string`：从左到右逐个取字符；

3. `if ch in "aeiouAEIOU"`：判断该字符是不是大小写元音之一；

4. 是元音则 `num += 1`；

5. 循环结束后 `return num`。

例如 `Hello` 中的元音是 `e` 和 `o`，所以返回 2。

4.8 案例 2：成绩最高分、最低分、平均分

修正后的可运行版本：

```
def calc_score(score_list):
    """Return max, min and average score (1 decimal place)."""
    max_s = max(score_list)
    min_s = min(score_list)
    avg_s = round(sum(score_list) / len(score_list), 1)
    return max_s, min_s, avg_s
```

Python 内置函数 `max()` 返回最大项，`min()` 返回最小项，`len()` 返回元素个数，`sum()` 求和，`round(number, ndigits)` 按给定小数位数舍入。[2]

平均分对应数学公式：

$$\bar{x} = \frac{x_1 + x_2 + \cdots + x_n}{n} = \frac{\sum_{i=1}^{n} x_i}{n}.$$

其中：

- x_i：第 i 个成绩；
- n：成绩数量，对应 `len(score_list)`；
- $\sum x_i$：总分，对应 `sum(score_list)`；
- $\bar{x}$：平均分。

使用前提：这里假设列表非空且元素都是可进行数值运算的成绩。

最简单示例

`score_list = [80, 90, 100]`：

$$\max = 100, \qquad \min = 80, \qquad \bar{x} = \frac{80 + 90 + 100}{3} = 90.0.$$

所以函数返回 `(100, 80, 90.0)`。

4.9 变量作用域：局部变量与全局变量

基本概念

变量作用域（scope）表示某个名字在程序的哪些位置可以被直接引用。MIT 课程把 scope 描述为“名字到对象绑定所在的上下文”；Python 官方 FAQ 进一步说明：函数体内仅被读取的名字可以引用全局变量，但如果函数体内对某名字赋值，默认把它视为局部名字，除非显式声明为 `global`。[3][6]

局部变量例子：

```
def f():
    x = 10
    print(x)

f()
# print(x)  # x is not directly available here
```

`x` 是在 `f()` 内赋值的局部变量。

全局变量例子：

```
pi = 3.14

def circle_area(r):
    return pi * r * r
```

函数只读取 `pi`，不需要写 `global pi`。

一句记忆：函数里新赋值的普通名字通常是局部；函数外顶层名字属于模块全局。

4.10 global 关键字

Python 官方参考说明，`global name` 使该名字在当前作用域中被解释为全局名字；尤其当函数内部要给全局名字赋值时，需要该声明。[3]

```
counter = 0

def add_one():
    global counter
    counter = counter + 1

add_one()
print(counter)
```

输出 `1`。

必须纠正的语法

错误：`global pi = 3.14`。

正确写法之一：

```
pi = 3.14

def change_pi():
    global pi
    pi = 3.1416
```

`global` 后只写名字；赋值是另一条语句。

初学者建议：能通过参数传入、通过 `return` 返回时，优先这样做；不要为了省几行代码大量修改全局变量。

5 高频易错点总结

1. `def` 行末漏掉冒号。
2. 函数体忘记缩进。
3. 函数定义处把“形参”叫成“实参”。
4. 调用时少传或多传参数。
5. 认为没有 `return` 就“什么也不返回”；实际上返回 `None`。[1]
6. 把 `print()` 当作 `return`。
7. docstring 没放在函数体第一条语句，却以为仍是 `__doc__`。
8. 成绩函数里变量名写成 `avg_s`，返回时却写 `avg`。
9. 误写 `global x = 1`；正确是 `global x` 后再单独赋值。
10. 仅仅读取全局变量时也机械地写 `global`；通常不需要。[3]

6 知识点—题目对应表

题号	题型	对应知识点	难度	主要能力
1	单选	K2：`def`	入门	语法识别
2	单选	K3：函数调用	入门	语法识别
3	单选	K4：形参与实参	入门	概念识别
4	单选	K5：返回值	入门	基本规则
5	单选	K6：docstring	入门	概念识别

题号	题型	对应知识点	难度	主要能力
6	单选	K7：嵌套调用	基础	顺序判断
7	单选	K8：元音统计	基础	代码阅读
8	单选	K8：内置统计函数	入门	函数识别
9	单选	K9：局部变量	入门	作用域识别
10	单选	K10：`global`	入门	语法识别
11	多选	K1–K5：函数基础	入门	规则辨认
12	多选	K6：docstring	入门	规则辨认
13	多选	K7：嵌套调用	基础	顺序判断
14	多选	K8：成绩统计	基础	代码理解
15	多选	K9–K10：作用域	基础	规则辨认
16	填空	K2：函数定义	入门	语法回忆
17	填空	K3：函数调用	入门	语法回忆
18	填空	K8：元音统计	入门	简单计算
19	填空	K8：平均分	基础	直接计算
20	填空	K10：`global`	入门	关键字回忆
21	解答	K2–K3	入门	定义并调用
22	解答	K4–K6	基础	参数、返回值、文档
23	解答	K8	基础	统计函数应用
24	解答	K9–K10	基础	修改全局计数器

7 配套练习

7.1 单项选择题（10 题）

每题只有一个正确答案。

1. 在 Python 中，用于开始定义函数的关键字是（　）。

A. function

B. def

C. return

D. print

2. 已经定义 `def out_line(): ...` 后，正确调用该函数的是（　）。

A. `def out_line()`

B. `out_line()`

C. `return out_line`

D. `print def out_line`

3. 在 `def add(a, b):` 中，`a` 和 `b` 属于（　）。

A. 实参

B. 形参

C. 返回值

D. 全局变量

4. 一个普通函数执行到末尾都没有写 `return`，它的返回值是（　）。

A. 0

B. 空字符串

C. None

D. False

5. 要让字符串成为函数的文档字符串 docstring，它最基本的位置应当是（　）。

A. 函数体第一条语句

B. 文件最后一行

C. 任意注释后面

D. `return` 之后

6. 运行下面代码时，最先输出的是（　）。

```
def a():
    print("A1")
```

```
    b()
    print("A2")

def b():
    print("B")

a()
```

A. A1

B. B

C. A2

D. None

7. 使用图片中的元音统计思路，`count_vowels("Hello")` 应返回（　）。

A. 1

B. 2

C. 3

D. 5

8. 要从列表 `[80, 90, 100]` 中直接得到最大值，最适合使用（　）。

A. `max()`

B. `min()`

C. `len()`

D. `round()`

9. 在函数内部写 `x = 10`，且没有 `global x`，这个 `x` 在最基础情况下属于（　）。

A. 局部变量

B. 必然是全局变量

C. 函数名

D. 文档字符串

10. 若函数内部确实要给模块全局变量 `counter` 重新赋值，正确的声明是（　）。

A. `global counter`

B. `global counter = 0`

C. `def global counter`

D. `return global counter`

7.2 多项选择题（5 题）

评分规则：每题有 2 个或 3 个正确选项；全部选对得分，漏选或错选均不得分。

11. 关于 Python 基础函数，下列说法正确的是（　）。

A. `def` 可用于定义函数

B. 函数可以没有形参

C. 函数必须至少写一个 `return`

D. 调用函数通常写成 `函数名(...)`

12. 关于 docstring，下列说法正确的是（　）。

A. 函数体第一条字符串语句可成为函数 docstring

B. PEP 257 推荐使用三重双引号

C. `:param` 是 Python 必须的语法关键字

D. docstring 可以说明函数功能、参数和返回值

13. 对下面嵌套调用代码，下列说法正确的是（　）。

```
def a():
    print("A-before")
    b()
    print("A-after")

def b():
    print("B")
```

A. 调用 `a()` 后会先打印 `A-before`

B. `b()` 返回后，`a()` 会继续执行

C. `A-after` 一定比 `B` 更早打印

D. 可用“进入 B、B 返回、继续 A”的方式理解

14. 关于 `calc_score(score_list)`，下列说法正确的是（　）。

A. `max(score_list)` 可求最高分

B. `min(score_list)` 可求最低分

C. `sum(score_list)/len(score_list)` 可求算术平均分

D. `round(x, 1)` 的作用是删除列表中的一个元素

15. 关于局部变量、全局变量和 `global`，下列说法正确的是（　）。

A. 函数只读取全局变量时，通常不必写 `global`

B. 函数内部若要给某全局名字重新赋值，可使用 `global name`

C. `global x = 1` 是规范的 Python 语法

D. 函数内部普通赋值产生的名字通常是局部变量

7.3 填空题（5 题）

16. 定义一个无参数函数 `hello` 的函数头应写为：_____。

17. 已定义无参数函数 `out_line`，调用它应写为：_____。

18. 字符串 `"AEiou"` 中共有 _____ 个元音字母。

19. 成绩 `[80, 90, 100]` 的平均分是 _____（写数值即可）。

20. 函数内部若要把名字 `counter` 声明为全局名字，应写：_____。

7.4 基础解答题（4 题）

21. **函数定义与调用**。定义一个无参数函数 `out_line()`，让它打印 10 个短横线 `----------`，然后调用一次该函数。

22. **参数、返回值与 docstring**。定义函数 `circle_area(r)`：输入半径 `r`，使用 `3.14 * r * r` 计算圆面积并返回；函数体第一条语句写一个简单 docstring。再求 `circle_area(2)` 的结果。

23. **成绩统计**。定义函数 `calc_score(score_list)`，对非空成绩列表求最高分、最低分和平均分，平均分保留 1 位小数，并一次返回三个结果。求 `calc_score([80, 90, 100])`。

24. **global 的最基础应用**。已知全局变量 `counter = 0`。定义函数 `add_one()`，每调用一次就让全局变量 `counter` 增加 1。调用一次后打印 `counter`。

8 答案速查表

1	2	3	4	5	6	7	8	9	10
B	B	B	C	A	A	B	A	A	A

11	12	13	14	15
ABD	ABD	ABD	ABC	ABD

题号	标准答案
16	`def hello():`
17	`out_line()`
18	5
19	90 或 90.0
20	`global counter`

9 单项选择题详细解析

第 1 题：答案 B

考查知识点： 函数定义关键字。
相关规则： Python 使用 `def` 引入函数定义，函数名与形参列表紧跟其后。[1]
判断过程： 题目问“开始定义函数”的关键字，只需识别语法。
A 错： `function` 不是 Python 定义普通函数的关键字。
B 对： `def` 正是函数定义关键字。
C 错： `return` 用于从函数返回结果，而不是开始定义函数。
D 错： `print` 用于输出内容。
常见误区： 受其他编程语言影响，把 `function` 当作 Python 关键字。
一句总结： 看到“定义函数”就先想到 `def`。

第 2 题：答案 B

考查知识点： 函数调用。
定义/规则： 函数定义完成后，最基本的调用形式是 *函数名（实参）*；无参数函数调用时括号仍然要写。
A 错： `def out_line()` 是函数定义开头，不是调用。
B 对： `out_line()` 是标准无参数调用形式。
C 错： `return` 只能在函数体中用于返回。
D 错： 不是合法 Python 语句。
易错点： 只写 `out_line` 得到的是函数对象本身，不等于执行函数。

一句总结：真正执行函数要带括号。

第 3 题：答案 B

考查知识点：形参与实参。
相关定义：`def add(a, b)`：中括号里的 `a`、`b` 是函数定义时的形式参数，即形参。[1]
A 错：实参是在调用时传入的具体值，例如 `add(2,3)` 中的 2 和 3。
B 对：`a`、`b` 是形参。
C 错：返回值由 `return` 产生。
D 错：形参在本次函数调用的局部环境中使用，不是因为写在 `def` 里就成为全局变量。
一句总结：定义处看“形参”，调用处看“实参”。

第 4 题：答案 C

考查知识点：无显式返回语句时的返回值。
核心规则：函数没有 `return`，或只写空 `return`，最终返回 `None`。[1]
A 错：不会自动返回 0。
B 错：不会自动返回空字符串。
C 对：标准返回值是 `None`。
D 错：`False` 与 `None` 是不同对象。
易错点：“没有写返回值”不等于“Python 没有任何返回对象”。
一句总结：没写 return，默认得到 None。

第 5 题：答案 A

考查知识点：docstring 的位置。
核心规则：函数体第一条语句如果是字符串字面量，它会成为函数的文档字符串。[1][4]
A 对：这是成为函数 `__doc__` 的关键位置要求。
B 错：文件末尾字符串与该函数 docstring 无直接关系。
C 错：任意位置的字符串不一定被识别为 docstring。
D 错：`return` 执行后函数已经退出，后面的普通语句通常也不会执行。
一句总结：docstring 要站在函数体“第一排”。

第 6 题：答案 A

考查知识点：嵌套调用执行顺序。
解题思路：调用 `a()` 后，从 `a` 的第一行函数体开始顺序执行。
步骤 1：`print("A1")` 先执行，所以第一条输出是 `A1`。
步骤 2：随后调用 `b()`，输出 `B`。
步骤 3：`b()` 返回，继续 `a()`，最后输出 `A2`。
完整输出顺序是 `A1 -> B -> A2`。
B、C 错：它们不是第一条输出。
D 错：`None` 是可能的返回值，不是代码中的第一条打印内容。
一句总结：先执行调用者中“调用之前”的语句。

第 7 题：答案 B

考查知识点：元音统计。
字符串 `Hello` 依次为 H、e、l、l、o。集合字符串 `aeiouAEIOU` 中包含 e 和 o，共 2 个。
A 错：漏数了一个元音。
B 对：e、o 共 2 个。
C 错：l 不是元音。
D 错：不是所有字符都是元音。
易错点：统计的是“字符出现次数”，不是“不同元音种类数”。
一句总结：逐字符检查，命中一次就加 1。

第 8 题：答案 A

考查知识点：常用内置函数。
`max(iterable)` 返回可迭代对象中的最大项；`min()` 返回最小项；`len()` 返回项目数；`round()` 用于舍入数值。[2]
因此要求 `[80,90,100]` 的最大值应使用 `max()`。
一句总结：最大 max，最小 min，个数 len，舍入 round。

第 9 题：答案 A

考查知识点：局部变量。
函数内部对名字 `x` 赋值，且没有 `global x`，Python 在最基础规则下把它视为该函数局部名字。官方 FAQ 明确说明：如果一个变量在函数体内被赋值，则默认认为它是局部的，除非显式声明为全局。[3]
B 错：不是“必然全局”。
C 错：`x` 是变量名，不是函数名。
D 错：文档字符串是字符串语句，不是普通数值赋值。
一句总结：函数里普通赋值，先按局部理解。

第 10 题：答案 A

考查知识点：`global` 的合法语法。
Python 参考语法为 `global identifier (, identifier)*`，即 `global` 后列出名字。[3]
A 对：`global counter` 合法。
B 错：`global` 声明不能与 `= 0` 直接写成一条赋值语句。
C、D 错：都不符合 Python 语法。
一句总结：global 只“声明名字”，赋值另写一行。

10 多项选择题详细解析

第 11 题：答案 ABD

A：正确。`def` 用于定义函数。[1]
B：正确。函数可以无参数，例如 `def out_line():`。

C：错误。函数不要求必须写 `return`；没有显式 `return` 时返回 `None`。[1]
D：正确。最基本调用形式为 `函数名(...)`。
汇总：ABD。
易错点：把“函数有返回值”误解成“源代码里必须出现 return”。

第 12 题：答案 ABD

A：正确。函数体第一条字符串字面量可成为 docstring。[1]
B：正确。PEP 257 推荐三重双引号。[4]
C：错误。`:param` 是 Sphinx/reStructuredText 常见文档字段，不是 Python 语言必须关键字。[8]
D：正确。PEP 257 建议函数 docstring 说明行为、参数、返回值等必要信息。[4]
汇总：ABD。

第 13 题：答案 ABD

A：正确。进入 `a()` 后，第一条语句就是打印 `A-before`。
B：正确。`b()` 完成后控制流回到 `a()` 调用点之后，继续执行。
C：错误。`A-after` 必须等 `b()` 返回后才执行，所以它晚于 `B`。
D：正确。“进入被调用函数—被调用函数返回—调用者继续”是适合本题的入门模型。[6][7]
汇总：ABD。

第 14 题：答案 ABC

A：正确。`max()` 返回最大项。[2]
B：正确。`min()` 返回最小项。[2]
C：正确。总和除以数量就是算术平均值；代码对应 `sum()/len()`。
D：错误。`round(x,1)` 是把数值舍入到 1 位小数附近的精度，不会删除列表元素。[2]
汇总：ABC。
易错点：`round` 处理的是数值，不是容器修改函数。

第 15 题：答案 ABD

A：正确。函数仅引用全局变量时通常可直接读取，不需声明 `global`。[3]
B：正确。如果函数内部要让赋值作用到全局绑定，可使用 `global name`。[3]
C：错误。`global x = 1` 不是合法语法。
D：正确。函数内部普通赋值的名字通常被视为局部名字。[3]
汇总：ABD。
一句总结：读全局通常不用 global；改全局绑定才需要它。

11 填空题详细解析

第 16 题

标准答案： `def hello():`

规则： 函数定义头由 `def` + 函数名 + 圆括号 + 冒号构成。因为本题无参数，所以括号中留空。

格式检查： 不能漏掉 `()`，也不能漏掉最后的冒号。

等价答案： 本题按指定函数名，只接受等价空格差异，例如 `def hello( ):`。

第 17 题

标准答案： `out_line()`

规则： 调用无参数函数仍要写圆括号。

易错点： `out_line` 只是引用函数对象；`out_line()` 才执行函数。

第 18 题

标准答案： 5。

推理： `AEiou` 五个字符分别为 A、E、i、o、u，都出现在 `aeiouAEIOU` 中，所以计数器连续增加 5 次。

单位/格式： 这是字符个数，没有物理单位。

等价答案： 5。

第 19 题

标准答案： 90 或 90.0。

使用公式： 算术平均数

$$\bar{x} = \frac{x_1 + x_2 + x_3}{3}.$$

x_1, x_2, x_3 分别是三个成绩，3 是成绩个数。

代入：

$$\bar{x} = \frac{80 + 90 + 100}{3} = \frac{270}{3} = 90.$$

若程序使用 `round(...,1)`，通常显示为 `90.0`。

易错点： 不要除以总分 270；应除以成绩个数 3。

第 20 题

标准答案： `global counter`。

规则： `global` 语句后写需要按全局名字解释的标识符。[3]

错误形式： `global counter = 0`。声明与赋值不能合并成这一写法。

等价答案： 允许多余空格，但语义必须是 `global counter`。

12 基础解答题详细解析

第 21 题：函数定义与调用

题意分析：已知要完成一个固定动作——打印 10 个短横线；函数不需要输入，也不要求返回计算值。
知识定位：`def` 定义无参数函数；`print()` 产生输出；`out_line()` 调用函数。
解题步骤：

1. 写函数头 `def out_line():`；
2. 缩进写 `print("----------")`；
3. 退出函数定义后写 `out_line()`。

标准代码：

```
def out_line():
    print("----------")

out_line()
```

运行结果：

```
----------
```

结果检查：确认恰好打印 10 个短横线，并且只打印一次。
易错提醒：函数体必须缩进；调用语句不要缩进到函数体里，否则会改变程序含义。
方法总结：固定重复动作最适合先封装成无参数函数，再按需调用。

第 22 题：参数、返回值与 docstring

题意分析：输入半径 r，输出圆面积；还要求函数体第一条语句是文档字符串。
知识定位：函数形参、docstring、`return`、圆面积公式。
公式说明：

$$S = \pi r^2.$$

本题按图片约定用 $\pi \approx 3.14$。r 是已知半径，S 是要求的面积。
标准代码：

```
def circle_area(r):
    """Return the area of a circle with radius r."""
    return 3.14 * r * r

result = circle_area(2)
print(result)
```

代入计算：

$$S = 3.14 \times 2 \times 2 = 12.56.$$

所以返回值为 `12.56`。
结果检查：半径为 2 时面积应大于 $2 \times 2 = 4$，12.56 的数量级合理。若半径单位是 cm，面

积单位是 cm^2。
易错提醒：不要写成 `3.14 * r * 2`；r^2 表示 $r \times r$。docstring 要位于函数体第一条语句。
方法总结：“输入参数—代入公式—return 结果”是最典型的计算函数结构。

第 23 题：成绩统计

题意分析：输入非空成绩列表，输出最高分、最低分、平均分；平均分保留 1 位小数。
知识定位：`max`、`min`、`sum`、`len`、`round` 与多值返回。[2]
平均分公式：

$$\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}.$$

x_i 是每个成绩，n 是成绩数量，`sum(score_list)` 对应总分，`len(score_list)` 对应 n。
标准代码：

```
def calc_score(score_list):
    max_s = max(score_list)
    min_s = min(score_list)
    avg_s = round(sum(score_list) / len(score_list), 1)
    return max_s, min_s, avg_s

print(calc_score([80, 90, 100]))
```

代入计算：

$$\max = 100, \quad \min = 80,$$
$$\bar{x} = \frac{80 + 90 + 100}{3} = 90.0.$$

规范结果：`(100, 80, 90.0)`。
结果检查：平均分必须位于最低分 80 与最高分 100 之间；90.0 满足这一基本检查。
易错提醒：图片中的 `return max_s, min_s, avg` 有变量名错误，必须返回 `avg_s`。列表不能为空。
方法总结：先分别得到三个统计量，再一次 `return` 即可。

第 24 题：global 的最基础应用

题意分析：全局已经存在 `counter=0`，函数要修改的正是这个全局绑定。
知识定位：`global` 声明与赋值。Python 官方参考指出，若函数内部需要把赋值解释为对全局名字的赋值，应先写 `global counter`。[3]
解题步骤：

1. 在模块顶层写 `counter = 0`；
2. 定义 `add_one()`；
3. 函数内先写 `global counter`；
4. 再写 `counter += 1`；
5. 调用一次并打印。

标准代码：

```python
counter = 0

def add_one():
    global counter
    counter += 1

add_one()
print(counter)
```

运行结果： 1。
结果检查： 初始值是 0，只调用一次，每次加 1，因此结果必须是 1。
易错提醒： 不能写 `global counter = 0`；也不要把 `counter = 0` 重新写进函数内部，否则会变成另一种逻辑。
方法总结： 需要“修改模块全局名字”时，先声明 `global name`，再单独赋值。

13 基础解题步骤总结

看到“定义一个函数”

1. 先确定函数名；
2. 看是否需要输入：需要就写形参，不需要就空括号；
3. 写 `def name(...):`；
4. 缩进写功能代码；
5. 题目要求“得到/返回结果”时考虑 `return`；
6. 在定义之后调用测试。

看到“求最大、最小、平均”

1. 最大：`max(list)`；
2. 最小：`min(list)`；
3. 总和：`sum(list)`；
4. 个数：`len(list)`；
5. 平均：`sum(list)/len(list)`；
6. 保留 1 位：`round(...,1)`。

看到“函数里修改全局变量”

1. 先确认它确实是模块顶层的全局名字；
2. 函数内部先写 `global name`；

3. 下一行再修改它；
4. 不要写 `global name = value`。

14 资料核验说明

核心结论进行了官方文档与高校课程材料交叉核验：函数定义、形参、docstring、默认返回 `None` 主要依据 Python 官方教程；`global` 与局部/全局规则依据 Python 语言参考和官方 FAQ；docstring 规范依据 PEP 257；嵌套调用与作用域教学模型参考 MIT OCW 与 UC Berkeley CS 61A；图片中的 `:param` / `:return` 样式使用 Sphinx 官方文档核验。[1][3][4][5][6][7][8]

15 参考资料与核验来源

[1] Python Software Foundation. *Python 3.13.15 Tutorial: More Control Flow Tools, Defining Functions*. Python Documentation. 访问日期：2026-08-20。https://docs.python.org/3.13/tutorial/controlflow.html

[2] Python Software Foundation. *Python 3.13.15 Built-in Functions*. Python Documentation. 访问日期：2026-08-20。https://docs.python.org/3.13/library/functions.html

[3] Python Software Foundation. *Programming FAQ: What are the rules for local and global variables in Python?*；以及 *Simple statements: global statement*. 访问日期：2026-08-20。https://docs.python.org/3/faq/programming.html；https://docs.python.org/3/reference/simple_stmts.html

[4] Goodger, D.; van Rossum, G. *PEP 257 – Docstring Conventions*. Python Enhancement Proposals. 2001，持续维护。访问日期：2026-08-20。https://peps.python.org/pep-0257/

[5] MIT OpenCourseWare. *6.0001 Lecture 4: Decomposition, Abstraction, and Functions*. Massachusetts Institute of Technology. Fall 2016. 访问日期：2026-08-20。https://ocw.mit.edu/courses/6-0001-introduction-to-computer-science-and-programming-in-python-fall-2016/resources/lecture-4-decomposition-abstraction-and-functions/

[6] MIT OpenCourseWare. *6.100L Lecture 8: Functions as Objects*. Massachusetts Institute of Technology. Fall 2022. 访问日期：2026-08-20。https://ocw.mit.edu/courses/6-100l-introduction-to-cs-and-programming-using-python-fall-2022/pages/lecture-8-functions-as-objects/

[7] UC Berkeley CS 61A. *Functions / Environments / Scope course materials*. University of California, Berkeley. 访问日期：2026-08-20。https://site.cs61a.org/lab/lab01/；https://cs61a.org/assets/slides/03-Higher-Order_Functions_1pp.pdf

[8] Sphinx Project. *The Python Domain – Info field lists*; *reStructuredText Primer*. 访问日期：2026-08-20。https://www.sphinx-doc.org/en/master/usage/domains/python.html；https://www.sphinx-doc.org/en/master/usage/restructuredtext/basics.html

16 质量检查记录

检查项	结果
图片正文重新编码	已按 4 页原始逻辑顺序整理；无法可靠辨认处已显式标注。
明显错误纠正	已纠正 `avg_s` 与 `avg` 变量名不一致、`global pi = 3.14` 非法语法、作用域表述过度简化等问题。
题量	10 单选 + 5 多选 + 5 填空 + 4 解答 = 24 题，数量核对完成。
难度	全部标为入门/基础；无竞赛、证明、复杂参数讨论和长计算。
答案一致性	单选唯一答案、多选 2–3 个正确选项、填空明确、解答代码与结果均已复核。
代码检查	核心示例在 Python 3.13 环境中运行验证。
来源	采用 Python 官方文档、PEP、MIT、UC Berkeley、Sphinx 官方文档；未使用内容农场作为核心依据。
PDF 编译	XeLaTeX 实际编译后再进行页面渲染检查。

本资料的目标是“第一次学完函数后，能看懂、能模仿、能完成最基础应用”。